overlay.py

User manual | Rigid-body least-squares superposition in PyMOL

Purpose. Define a structural fit using selected atoms from two different PyMOL molecular objects, then move the entire mobile parent object onto the fixed target object.

Scope. Protein C-alpha correspondences; named heavy-atom correspondences for one non-protein residue; optional transformation reports and CGO arrow commands.

1. Requirements and installation

- PyMOL running under Python 3, with NumPy available within PyMOL's Python environment.
- Two different PyMOL molecular objects, each having exactly one coordinate state.
- Optional `cgo_arrow.py` script, only to display arrow commands supplied in the report.

Load both molecular objects into PyMOL. Load the script using the PyMOL command line:

```
run /path/to/overlay.py
```

If the file is in the current working directory, use `run overlay.py`. The script registers the `overlay` command using `cmd.extend`.

2. Basic usage

```
overlay mobile, target [, mode=strict|ordered [, report=auto|0|filename]]
```

The first argument identifies fitting atoms within the object to move; the second identifies fitting atoms in the fixed reference object. Selections must resolve to exactly one object each. The script validates the atom pairs, calculates a Kabsch rigid-body fit, and transforms the entire mobile object.

Argument	Meaning
`mobile`	Selection of fitting atoms in the object that moves
`target`	Selection of fitting atoms in the fixed reference object
`mode=strict`	Default; matching by protein residue IDs or ligand atom names
`mode=ordered`	Single-chain protein only; matched by sorted position, not sequence
`report=auto`	Default; automatically named text report in working directory
`report=0`	Disable report; off/none/no/false also accepted
`report=filename`	Output text report at chosen path

3. Worked command patterns

Protein selections

```
select mobile_site, mobile_obj and chain A and resi 10-80
select target_site, target_obj and chain B and resi 10-80
overlay mobile_site, target_site
```

With one selected protein chain on each side, strict correspondence uses matching residue identifiers, even if the chain letters differ. The selected regions need not include the whole object: all atoms of `mobile_obj` will move.

Single-chain protein with different residue numbering

```
overlay mobile_site, target_site, mode=ordered
```

Use only when the selected, sorted C-alpha atoms are known to correspond in sequence position. Both sides must contain the same number of C-alpha atoms. No sequence identity check is performed.

Single-residue ligand

```
select moving_lig, mobile_obj and resn LIG
select fixed_lig, target_obj and resn LIG
overlay moving_lig, fixed_lig, report=ligand_fit.txt
```

The ligand match requires one residue in each compared selection, at least three heavy atoms, identical unique heavy-atom names, and matching elements.

Fit without a text report

```
overlay mobile_site, target_site, report=0
```

4. How atom correspondence is determined

Protein rules

- The script identifies proteins using PyMOL's `polymer.protein` classification and fits selected atoms named `CA`.
- At least three fitting C-alpha atoms must be selected, without duplicate segment/chain/residue identifiers.
- For one selected chain per side, strict mode requires identical residue identifiers; chain names may differ.
- For multiple chains, only strict mode is supported, and complete segment/chain/residue keys must match between mobile and target.
- C-alpha atoms are sorted by segment, chain and residue number (including a possible insertion code).

Ligand and nonprotein rules

- Non-protein selections must resolve to exactly one residue, excluding water and nucleic polymer selections.
- Hydrogen atoms are omitted; at least three heavy atoms are needed.
- Unique atom names, exactly matching name sets and matching element symbols are required.
- Ordered mode is not supported for ligands.

5. Least-squares calculation and whole-object movement

For N paired atoms, the Kabsch calculation finds a proper rotation R and translation t minimizing the sum of squared distances between transformed mobile coordinates and corresponding target coordinates:

$$\mathbf{x}_{\text{target}} \approx R\mathbf{x}_{\text{mobile}} + \mathbf{t}$$

$$\text{RMSD} = \sqrt{\frac{1}{N}\sum_{i=1}^{N} \|R\mathbf{x}_i + \mathbf{t} - \mathbf{y}_i\|^2}$$

The implementation centers the paired coordinates, applies singular value decomposition, and corrects an improper reflection when necessary. It then sends a row-major 4×4 homogeneous matrix to PyMOL using `cmd.transform_object` for the mobile object. The fitting selections define the motion; they do not restrict which atoms move.

The RMSD shown and recorded is calculated from the fitted coordinates before PyMOL applies the object transformation. The script does not re-read the transformed coordinates to check the result independently.

6. The text report

By default, the file is named `overlay_<mobile-object>_onto_<target-object>.txt`. The name is based on PyMOL object names, not selection names. A custom filename may be given as the report option.

- Input selection strings, object names, molecular type, matching mode and number of atom pairs.
- Fitted RMSD in Ångstroms, 3×3 rotation matrix, 3-component translation and 4×4 homogeneous transformation.
- Rotation angle, screw-axis direction, signed axial displacement and translation magnitude (as applicable).
- Commands for illustrating the fit with CGO arrows, provided separately for manual use.

Understanding the screw description

For a nonzero rotation, an axis direction and the displacement along that axis are reported. For negligible rotation, the transformation is treated as a pure translation and has no unique rotation axis. A 180-degree rotation also has an inherent sign ambiguity for the axis direction.

COM refers to the unweighted geometric centroid, not a true molecular centre of mass.

7. Visualizing the movement

The text report includes CGO arrow commands for visualizing the fitted rigid-body transformation. Depending on the geometry of the transformation, these may illustrate centroid displacement, the screw axis, axial translation, and auxiliary projected displacements. Arrows corresponding to negligible displacements are omitted. To create the arrows, load the `cgo_arrow.py` script separately and paste the commands into the PyMOL command line. These arrows are not created automatically by `overlay.py`.

8. Troubleshooting

Symptom / message	What to check
One object per selection	Avoid selecting atoms from multiple PyMOL objects in the same fitting selection.
Protein residue mismatch	Check missing/extra residue numbers or choose ordered mode only for known single-chain positional matches.
Different selected chain sets	For multiple chains, match segment and chain IDs on both sides.
Ligand atom-name mismatch	Verify names, elements and one-residue selection; ligand matching does not infer connectivity.
Degenerate fitting geometry	Choose three or more non-collinear fitting atoms.
Multiple coordinate states	Prepare single-state objects; the script rejects multistate objects.
Report not written	Check writable output location or use `report=0`; the fit may still succeed.
Arrows do not work	Load a suitable `cgo_arrow.py`; arrows are optional and separate.

9. Limitations, reproducibility and publication

- Only rigid-body motion is fitted; there is no iterative outlier removal or flexible molecular alignment.
- The script does not automatically establish sequence alignment or chemical atom equivalence beyond its matching rules.
- Only single-state objects are accepted.